

OSSP Skill Week 6

Part 1: Backslash Escaping and Special Character Processing

Aim

To study how the Linux shell uses backslash escaping and quoting to preserve spaces and special characters, and to validate how arguments are parsed.

Commands

1. Create a working folder

```
mkdir week6_s  
cd ~/week6_s  
pwd
```

2. Basic backslash escaping

```
echo "Hello\nWorld"  
echo -e "Hello\nWorld"
```

3. Escaped spaces

```
echo Hello\ World  
echo Hello World
```

4. Escaping special characters

```
echo \  
echo \  
echo \
```

5. Preserving characters with quotes

```
echo '$HOME'  
echo "$HOME"
```

6. Validating parser output (each argument shown inside < >)

```
printf '<%s>\n' Hello\ World  
printf '<%s>\n' "Hello World"  
printf '<%s>\n' 'Hello World'
```

7. Complex input

```
echo "Hello\ World \$USER \& \*"
```

Output

```
cd: not a directory: /Users/hrushika/Downloads/pjsdj/week6_s
● hrushika@Hrushikas-MacBook-Air pjsdj % echo "Hello\nWorld"<2028>echo -e "Hello\nWorld"<2028>
Hello
World
World
● hrushika@Hrushikas-MacBook-Air pjsdj % echo Hello\ World<2028>echo Hello World<2028>
Hello World
Hello World
● hrushika@Hrushikas-MacBook-Air pjsdj % echo \*<2028>echo \&<2028>echo \|<2028>
* echo & echo |
● hrushika@Hrushikas-MacBook-Air pjsdj % echo '$HOME'<2028>echo "$HOME"
$HOME
$HOME echo /Users/hrushika
● hrushika@Hrushikas-MacBook-Air pjsdj % printf '%s>\n' Hello\ World<2028>printf '<s>\n' "Hello World"<2028>printf '<s>\n' 'Hello World'
<Hello World printf>
<<s>\n>
<Hello World printf>
<<s>\n>
<Hello World>
○ hrushika@Hrushikas-MacBook-Air pjsdj %
● hrushika@Hrushikas-MacBook-Air pjsdj % echo "Hello\ World \$USER \& \*"
Hello\ World $USER \& \*
○ hrushika@Hrushikas-MacBook-Air pjsdj %
```

Result

Backslash escaping and quoting were shown to control how the shell interprets special characters and preserves the intended input.

Part 2: Child Processes

Aim

To create a child process with `fork()`, execute commands with and without arguments using `exec()`, and manage the child from the parent using `wait()`.

Program 1: Executing a Command

```
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/wait.h>

int main(void)
{
    pid_t pid = fork();

    if (pid < 0) {
        perror("fork");
        return 1;
    }

    if (pid == 0) {
        char *args[] = { "ls", "-l", NULL };

        printf("Child Process: PID = %d\n", getpid());
        printf("Executing command: ls\n");
        fflush(stdout);

        execvp(args[0], args);
        perror("execvp");
        exit(1);
    }

    printf("Parent Process: PID = %d\n", getpid());
    printf("Waiting for child process...\n");

    if (waitpid(pid, NULL, 0) < 0) {
        perror("waitpid");
        return 1;
    }

    printf("Child process completed.\n");
    return 0;
}
```

Program 2: Passing Arguments to a Child Process

```
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/wait.h>

int main(void)
{
    char *args[] = { "echo", "Hello", "OSSP", "Week", "6", NULL };

    pid_t pid = fork();

    if (pid < 0) {
        perror("fork");
        return 1;
    }

    if (pid == 0) {
        printf("Child Process: PID = %d\n", getpid());
        printf("Running:");
        for (int i = 0; args[i] != NULL; i++)
            printf(" %s", args[i]);
        printf("\n");
        fflush(stdout);

        execvp(args[0], args);
        perror("execvp");
        exit(1);
    }

    printf("Parent Process: PID = %d\n", getpid());

    if (waitpid(pid, NULL, 0) < 0) {
        perror("waitpid");
        return 1;
    }

    printf("Child process completed.\n");
    return 0;
}
```

Output

```
hrushika@Hrushikas-MacBook-Air pjsdj % ./child_process
Parent Process: PID = 64232
Waiting for child process...
Child Process: PID = 64233
Executing command: ls
total 2016
-rw-r--r--@ 1 hrushika  staff   76 Oct  5 10:08 File1.txt
-rw-r--r--@ 1 hrushika  staff   76 Oct  5 10:09 File2.txt
-rwxr-xr-x@ 1 hrushika  staff 33928 Oct  6 16:10 a.out
-rwxr-xr-x@ 1 hrushika  staff 34184 Oct  6 17:06 builtin
-rw-r--r--@ 1 hrushika  staff  1596 Oct  6 17:04 builtin.c
-rwxr-xr-x@ 1 hrushika  staff 33968 Oct  6 16:58 cd
-rw-r--r--@ 1 hrushika  staff   976 Oct  6 16:57 cd.c
-rw-r--r--@ 1 hrushika  staff   772 Oct  6 17:25 child_args.c
-rwxr-xr-x@ 1 hrushika  staff 33920 Oct  6 17:26 child_process
-rw-r--r--@ 1 hrushika  staff   787 Oct  6 17:24 child_process.c
-rwxr-xr-x@ 1 hrushika  staff 33696 Oct  6 16:35 double_quote
-rw-r--r--@ 1 hrushika  staff  1481 Oct  6 16:35 double_quote.c
-rwxr-xr-x@ 1 hrushika  staff 33432 Oct  6 16:13 escape
-rw-r--r--@ 1 hrushika  staff   150 Oct  6 16:12 escape.c
-rwxr-xr-x@ 1 hrushika  staff 33688 Oct  6 16:42 escape1
-rw-r--r--@ 1 hrushika  staff   533 Oct  6 16:42 escape1.c
-rwxr-xr-x@ 1 hrushika  staff 33688 Oct  6 16:46 execute
-rw-r--r--@ 1 hrushika  staff   912 Oct  6 16:46 execute.c
-rwxr-xr-x@ 1 hrushika  staff 34024 Oct  6 17:15 export
-rw-r--r--@ 1 hrushika  staff  1362 Oct  6 17:15 export.c
-rwxr-xr-x@ 1 hrushika  staff 33944 Oct  5 09:39 fork.exec
-rw-r--r--@ 1 hrushika  staff   935 Oct  5 09:39 fork.exec.c
-rwxr-xr-x@ 1 hrushika  staff 33864 Oct  5 09:46 main_loop
-rw-r--r--@ 1 hrushika  staff   618 Oct  5 09:46 main_loop.c
-rwxr-xr-x@ 1 hrushika  staff 33688 Oct  6 16:28 parser
-rw-r--r--@ 1 hrushika  staff   806 Oct  6 16:28 parser.c
-rwxr-xr-x@ 1 hrushika  staff 33992 Oct  6 17:09 pwd_exit
-rw-r--r--@ 1 hrushika  staff   971 Oct  6 17:09 pwd_exit.c
-rwxr-xr-x@ 1 hrushika  staff 33640 Oct  6 16:33 single_quote
-rw-r--r--@ 1 hrushika  staff   546 Oct  6 16:32 single_quote.c
-rwxr-xr-x@ 1 hrushika  staff 33928 Oct  6 16:10 sk3t3
-rw-r--r--@ 1 hrushika  staff  1637 Oct  6 16:10 sk3t3.c
-rwxr-xr-x@ 1 hrushika  staff 33768 Oct  5 10:06 task2
-rw-r--r--@ 1 hrushika  staff   875 Oct  5 10:06 task2.c
-rwxr-xr-x@ 1 hrushika  staff 33928 Oct  5 10:12 task3
-rw-r--r--@ 1 hrushika  staff  1638 Oct  5 10:11 task3.c
-rwxr-xr-x@ 1 hrushika  staff 33944 Oct  6 16:19 task5
-rw-r--r--@ 1 hrushika  staff  3052 Oct  6 16:19 task5.c
-rwxr-xr-x@ 1 hrushika  staff 53960 Oct  6 16:20 task5_asan
drwxr-xr-x@ 3 hrushika  staff   96 Oct  6 16:20 task5_asan.dSYM
-rw-r--r--@ 1 hrushika  staff  4552 Sep 28 11:04 w10q1.c
-rw-r--r--@ 1 hrushika  staff  1479 Sep 28 11:08 w10q11.sh
-rwxr-xr-x@ 1 hrushika  staff 34520 Sep 28 11:15 w10q2
-rw-r--r--@ 1 hrushika  staff  6046 Sep 28 11:14 w10q2.c
-rwxr-xr-x@ 1 hrushika  staff 34040 Sep 28 11:24 w11q1
-rw-r--r--@ 1 hrushika  staff  2924 Sep 28 11:24 w11q1.c
-rwxr-xr-x@ 1 hrushika  staff 34040 Sep 28 11:29 w11q2
-rw-r--r--@ 1 hrushika  staff  4045 Sep 28 11:29 w11q2.c
-rwxr-xr-x@ 1 hrushika  staff 34616 Sep 28 10:43 w9q1
-rw-r--r--@ 1 hrushika  staff  6027 Sep 28 10:43 w9q1.c
-rwxr-xr-x@ 1 hrushika  staff 34424 Sep 28 10:53 w9q2
-rw-r--r--@ 1 hrushika  staff  4506 Sep 28 10:52 w9q2.c
-rw-r--r--@ 1 hrushika  staff    0 Oct  6 17:19 week6_s
Child process completed.
hrushika@Hrushikas-MacBook-Air pjsdj %
```

```
Child process completed.
hrushika@Hrushikas-MacBook-Air pjsdj % gcc child_args.c -o child_args
hrushika@Hrushikas-MacBook-Air pjsdj % ./child_args
Parent Process: PID = 64392
Child Process: PID = 64393
Running: echo Hello OSSP Week 6
Hello OSSP Week 6
Child process completed.
hrushika@Hrushikas-MacBook-Air pjsdj %
```

Observation

The child process executed `ls -l` and, in the second program, `echo` with four arguments, while the parent waited for it to finish. The final `ps` check showed no leftover child processes.

Result

Process creation and execution were implemented using `fork()`, `exec()` and `wait()`, with arguments passed to the child and execution errors handled using `perror()`.